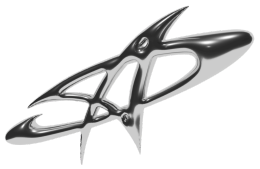
Week 06 · Masking & Decoder Block

미래를 훔쳐보지 않도록, 그리고 원문을 계속 참고하도록

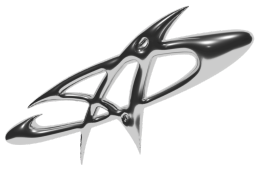
Padding Mask와 Look-ahead Mask로 Attention이 봐서는 안 되는 위치를 가리고, Cross-Attention으로 Decoder가 Encoder의 출력을 참고하게 만듭니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

이론 세션

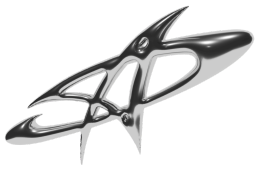
Decoder에게 필요한 두 가지 규칙



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Encoder는 입력 문장 전체를 한꺼번에 봐도 문제가 없었다.
하지만 Decoder는 “다음 단어를 하나씩 순서대로 생성”하는 역할이다.

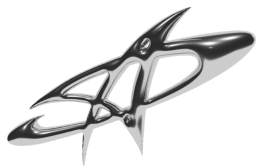
그래서 Decoder의 Attention에는 두 가지 규칙(마스크)이 추가로 필요하다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

“보지 말아야 할 곳”을 가리는 두 가지 경우

- **Padding Mask** — 배치 안에서 문장 길이를 맞추려고 채운 <pad> 토큰은 Attention 계산에서 **완전히 무시**해야 함
- **Look-ahead Mask(Causal Mask)** — Decoder가 **t번째 단어를 예측**할 때, **t+1번째 이후의 정답 단어를 미리 보면 반칙** (학습 때는 정답 시퀀스를 한꺼번에 넣으므로 반드시 가려야 함)
- 두 마스크 모두 방법은 동일: **가리고 싶은 위치의 attention score를 −inf로 만들어서, softmax를 거치면 0이 되게** 한다



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Padding Mask

짧은 문장 뒤의 <pad>는 없는 셈 치기

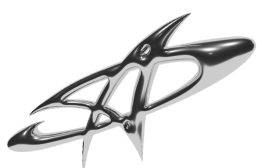
나는	밥을	먹었다	<pad>	<pad>

← 실제 길이 3, 배치 최대길이 5 맞춤용 패딩

1	1	1	0	0

← mask: 1=보임, 0=가림(−inf 처리)

실제 토큰 위치는 1, <pad> 위치는 0으로 표시한 마스크를 만들어 attention score에 적용합니다.



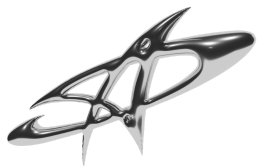
이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Look-ahead (Causal) Mask

미래 위치는 삼각형 모양으로 가린다

	나는	밥을	먹었다	맛있게	.
나는	O	X	X	X	X
밥을	O	O	X	X	X
먹었다	O	O	O	X	X
맛있게	O	O	O	O	X
.	O	O	O	O	O

행(row)=지금 예측하려는 위치, 열(col)=참고할 수 있는 위치. “먹었다”를 예측할 땐 “나는, 밥을, 먹었다”까지만 보이고 그 뒤는 가려진다(X).



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

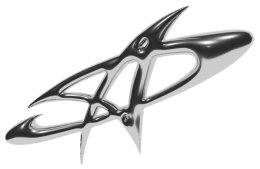
마스크를 실제로 적용하는 방법

가리고 싶은 자리를 -inf로

```
scores = scores.masked_fill(mask == 0, -∞)
weights = softmax(scores)
```

-∞를 쓰는 이유 — softmax(−∞) = 0 이 되어, 그 위치의 Value는 가중합에 전혀 기여하지 못함

왜 값을 지우지 않고 -inf로 바꾸는가 — 행렬 shape을 그대로 유지해야 배치 연산이 깨지지 않기 때문

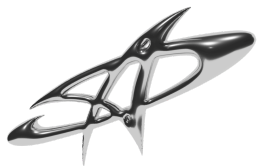


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Decoder Block의 3단계

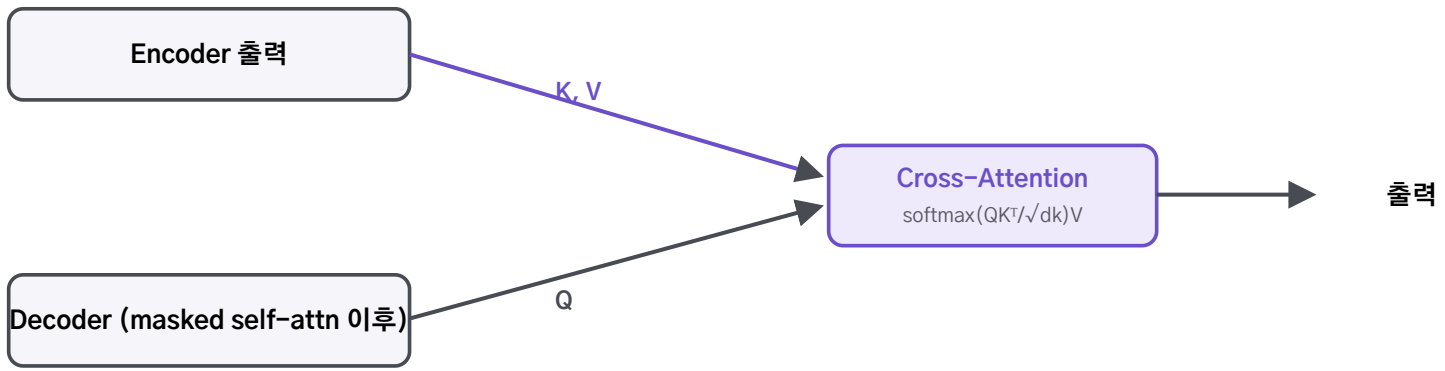
Encoder Block보다 서브레이어가 하나 더 있다

- **1. Masked Multi-Head Self-Attention** — 지금까지 생성된 단어들끼리만, look-ahead mask를 적용해서 Attention
- **2. Multi-Head Cross-Attention** — Query는 Decoder에서, **Key와 Value는 Encoder의 출력**에서 가져옴
- **3. Feed Forward Network** — Encoder와 동일한 position-wise FFN
- 세 서브레이어 **모두** Residual Connection + Layer Norm이 붙음 (5주차에서 만든 패턴 그대로)

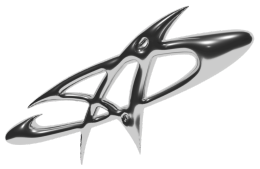


이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Decoder가 Encoder의 출력을 “참고”하는 방법



번역 작업으로 치면, Decoder가 한 단어씩 출력을 만들면서 계속 원문(Encoder 출력)을 다시 들여다보는 것과 같다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Decoder Block 조립도

세 개의 서브레이어, 각각에 Add&Norm



Encoder Block과 거의 같지만,
맨 앞에 “미래를 가린” Self-Attention이
있고, 그 다음 Encoder 출력을 보는
Cross-Attention이 끼어 있습니다.

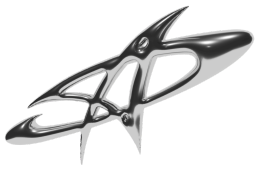
(세 번째 Add&Norm은 생략 —
FFN 뒤에도 동일하게 반복)

Masked Self-Attention → Cross-Attention → FFN, 이 세 서브레이어를 N번 쌓은 것이 Decoder 전체입니다.

이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

코딩 세션

마스크 생성 함수 +
DecoderLayer & Decoder Stack



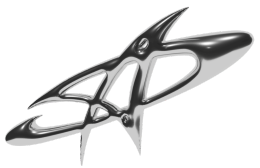
이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

<pad> 토큰 위치를 0으로

01_padding_mask.py

```
def make_padding_mask(seq, pad_idx=0):
    # seq: [batch, seq_len] 토큰 id
    mask = (seq != pad_idx).unsqueeze(1).unsqueeze(2)
    return mask   # [batch, 1, 1, seq_len] — attention score에 브로드캐스팅됨

seq = torch.tensor([[5, 8, 3, 0, 0]])
print(make_padding_mask(seq))
# tensor([[[[ True,  True,  True, False, False]]]])
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

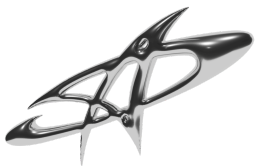
torch.triu로 삼각 행렬 만들기

02_causal_mask.py

```
def make_causal_mask(seq_len):
    # 대각선 포함 아래쪽(과거+현재)만 True
    mask = torch.tril(torch.ones(seq_len, seq_len)).bool()
    return mask   # [seq_len, seq_len]

print(make_causal_mask(4))
# tensor([[ True, False, False, False],
#         [ True,  True, False, False],
#         [ True,  True,  True, False],
#         [ True,  True,  True,  True]])
```

실전에서는 padding mask와 causal mask를 **AND(&) 연산**으로 합쳐서 한 번에 적용합니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

Cross-Attention을 위한 MHA 일반화

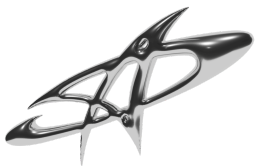
Q와 K,V의 출처를 분리하기

03_mha_generalized.py

```
# 4주차 버전은 x 하나에서 Q,K,V를 모두 만들었지만,
# Cross-Attention은 Q와 K,V의 출처가 다르므로 인자를 분리합니다.
def forward(self, query, key, value, mask=None):
    B = query.size(0)
    Q = self.split_heads(self.W_q(query))
    K = self.split_heads(self.W_k(key))
    V = self.split_heads(self.W_v(value))

    out, _ = scaled_dot_product_attention(Q, K, V, mask)
    out = out.transpose(1, 2).contiguous().view(B, -1, self.h * self.d_k)
    return self.W_o(out)
```

Self-Attention은 그냥 **query=key=value=x**로 호출하면 됩니다 — 완전히 같은 클래스를 재사용합니다.



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

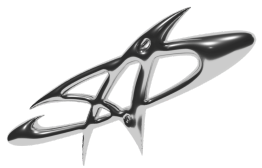
DecoderLayer 구현

Masked Self-Attn → Cross-Attn → FFN

04_decoder_layer.py

```
class DecoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads)
        self.cross_attn = MultiHeadAttention(d_model, num_heads)
        self.ffn = PositionwiseFeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, enc_out, causal_mask, pad_mask):
        x = self.norm1(x + self.dropout(self.self_attn(x, x, x, causal_mask)))
        x = self.norm2(x + self.dropout(self.cross_attn(x, enc_out, enc_out, pad_mask)))
        x = self.norm3(x + self.dropout(self.ffn(x)))
        return x
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

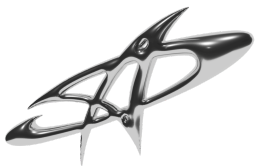
Decoder Stack + 더미 테스트

N개를 쌓고, Encoder 출력과 함께 forward

- Decoder도 Encoder처럼 **DecoderLayer를 N개** nn.ModuleList로 쌓기만 하면 됨
- 단, 매 layer마다 **같은 enc_out**을 계속 Cross-Attention에 전달

```
dec = nn.ModuleList([DecoderLayer(512,8,2048) for _ in range(6)])
x = torch.randn(2, 7, 512)        # decoder 입력
enc_out = torch.randn(2, 10, 512)  # encoder 출력
cmask = make_causal_mask(7)

for layer in dec:
    x = layer(x, enc_out, cmask, None)
print(x.shape)  # [2, 7, 512]
```



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지

6주차 정리

오늘 배운 것 & 다음 주 예고

오늘 배운 것

→Padding Mask = <pad> 무시, Causal Mask = 미래 차단

→masked_fill(mask==0, −inf) 후 softmax

→Cross-Attention: Q=Decoder, K,V=Encoder 출력

→DecoderLayer = Self-Attn + Cross-Attn + FFN

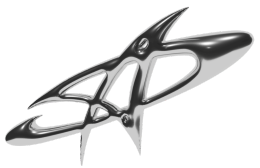
7주차 예고

→Encoder + Decoder를 하나로 합쳐 전체 Transformer 완성

→Output Linear + Softmax로 다음 단어 확률 만들기

→파라미터 수를 직접 세어보기

→원 논문(Attention Is All You Need) 리뷰



이 자료의 모든 저작권은 조치연에게 있습니다 · 무단 도용 금지